confirmation that the consumer wants to receive it. The flow is reversed in this sense. Consumers—also called *downstreams*—set the flow in motion. They have to signal to the producers—called *upstreams*—that they can process data so the producer can send data downstream (see figure 10.1).

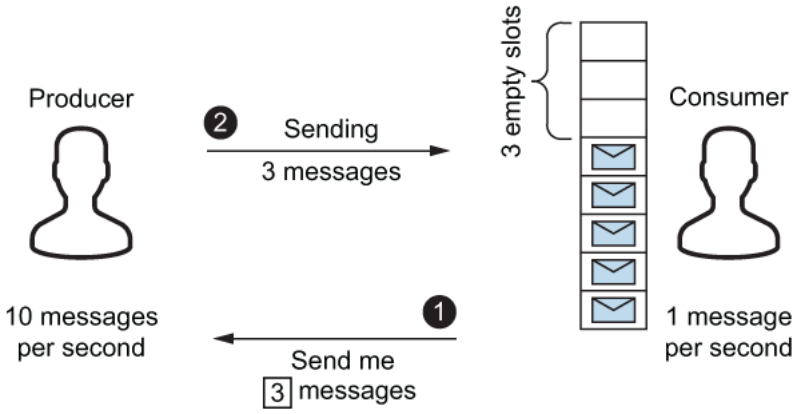


Figure 10.1 Consumer signaling that it's ready to consume three messages

10.1.1 Basic semantics

The basic abstractions of Akka Streams are the source, sink, and flow, which can be viewed respectively as a publisher, a consumer, and a combination of the two (see figure 10.2). The data flow goes from upstream to downstream, from source to sink, usually with a conversion in between.

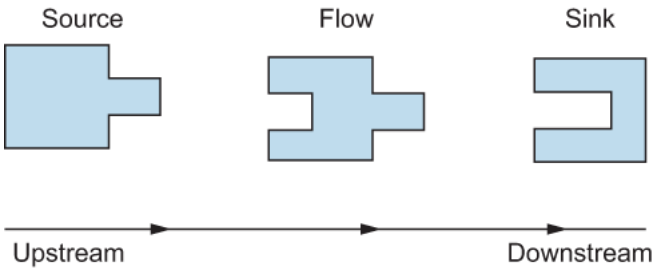


Figure 10.2 Akka Streams abstraction flow

You first need to import the Streams library, whether you're working with finite or infinite streams. Add the following to your build.sbt:

```
val AkkaVersion = "2.6.20"
libraryDependencies += "com.typesafe.akka" %% "akka-stream" % AkkaVersion
```

10.1.2 Finite streams

Let's start with finite data streams, which are easier to understand than infinite ones (the API is the same). Suppose you have a finite data source and you want to filter only the even values and store the